

Contents

1 General 1

1.1 C++ Template	1
1.2 C++ Visual Studio Includes	2
1.3 Python Template	2
1.4 Compilation	2
1.5 Automatic Test	2
2 Data Structures 2	
2.1 Disjoin Set Union	2
2.2 Fenwick Tree Point Update And Range Query	3
2.3 Fenwick Tree Range Update And Point Query	3
2.4 Fenwick Tree Range Update And Range Query	3
2.5 Fenwick 2D	3
2.6 Segment Tree	3
2.7 Segment Tree With Lazy Propagation	4
2.8 Treap	4
2.9 Implicit Treap	5
2.10 Trie	5
3 Graphs 6	
3.1 Dfs With Timestamps	6
3.2 Lowest Common Ancestor	6
3.3 Strongly Connected Components ...	6
3.4 Bellman Ford Algorithm	6
3.5 Bipartite Graph	7
3.6 Finding Articulation Points	7
3.7 Finding Bridges	8
3.8 Max Flow With Ford Fulkerson ...	8
3.9 Max Flow With Dinic	8
3.10 Max Flow With Dinic 2	9
3.11 Min Cut	9
3.12 Number Of Paths Of Fixed Length ..	10
3.13 Shortest Paths Of Fixed Length	10
3.14 Dijkstra	10
4 Geometry 10	
4.1 2d Vector	10
4.2 Line	10
4.3 Convex Hull Gift Wrapping	10
4.4 Convex Hull With Graham's Scan ..	11
4.5 Circle Line Intersection	11
4.6 Circle Circle Intersection	11
4.7 Common Tangents To Two Circles ..	11
4.8 Number Of Lattice Points On Segment	11
4.9 Pick's Theorem	12
4.10 Usage Of Complex	12
4.11 Misc	12
5 Math 13	

5.1 Linear Sieve	13
5.2 Extended Euclidean Algorithm	13
5.3 Chinese Remainder Theorem	13
5.4 Euler Totient Function	13
5.5 Factorization With Sieve	13
5.6 Modular Inverse	14
5.7 Simpson Integration	14
5.8 Burnside's Lemma	14
5.9 FFT	14
5.10 FFT With Modulo	15
5.11 Big Integer Multiplication With FFT .	15
5.12 Gaussian Elimination	16
5.13 Sprague Grundy Theorem	16
5.14 Formulas	16
6 Strings 16	
6.1 Hashing	16
6.2 Prefix Function	17
6.3 Prefix Function Automaton	17
6.4 KMP	17
6.5 Aho Corasick Automaton	17
6.6 Suffix Array	18
7 Dynamic Programming 18	
7.1 Convex Hull Trick	18
7.2 Divide And Conquer	19
7.3 Optimizations	19
8 Misc 19	
8.1 Mo's Algorithm	19
8.2 Ternary Search	19
8.3 Big Integer	19
8.4 Binary Exponentiation	21
8.5 Builtin GCC Stuff	21
1 General	

GENERAL CODE SYNTAX

1.Binary search(upper and lower bound)

```
vector<int> v = {10, 20, 30, 30, 40};
int x = 30;

// First element >= x
auto lb = lower_bound(v.begin(), v.end(), x);
int idx_lb = lb - v.begin(); // Index: 2

// First element > x
auto ub = upper_bound(v.begin(), v.end(), x);
int idx_ub = ub - v.begin(); // Index: 4

// Count occurrences of x in O(log N)
int count = ub - lb; // Result: 2

// Check if x exists
bool exists = binary_search(v.begin(), v.end(), x);
```

2.Built-in Bit Manipulation (GCC)

```
unsigned int n = 7; // Binary: 111

__builtin_popcount(n);    // Count set bits (1s).
Result: 3

__builtin_clz(n);         // Count leading zeros
(starts from most significant bit)

__builtin_ctz(n);         // Count trailing zeros
(starts from least significant bit)

__builtin_parity(n);      // 1 if number of set bits is
odd, 0 if even

// For long long, append 'll'
__builtin_popcountll(n_ll);
```

3.Floating point precision

```
const double EPS = 1e-9;
if (abs(a - b) < EPS) {
    // a and b are effectively equal
}
```

```
// Print with fixed precision
cout << fixed << setprecision(10) << ans << endl;
```

4.String and character utilities

```
// Check character types
isalpha(c); isdigit(c); isalnum(c); islower(c);
isupper(c);
```

```
// Convert case
char lower = tolower(c);
```

```
// String to Number / Number to String
int x = stoi("123");
string s = to_string(123);
```

6.Custom sort with lambdas

```
sort(v.begin(), v.end(), [](const Type &a, const
Type &b) {
    if (a.value != b.value) return a.value < b.value;
    return a.secondary < b.secondary; // Tie-breaker
});
```

7.Character to number and vice versa

```
char c = '7';
int num = c - '0';    // Convert '7' to 7
char back = num + '0'; // Convert 7 back to '7'
char letter = 'd';
int index = letter - 'a'; // 'a'->0, 'b'->1, 'c'->2, 'd'->3
```

```
// Convert index back to character
char original = index + 'a';
```

```
// For Uppercase
char cap = 'G';
int cap_idx = cap - 'A';
```

8. Setting and unsetting bits etc

Set i-th bit `n | (1 << i)` Uses OR to force the bit to 1.

Unset (Clear) i-th bit `n & ~(1 << i)` Uses AND with a mask of all 1s except at position i.

Toggle i-th bit `n ^ (1 << i)` Uses XOR to flip the bit (0 to 1, or 1 to 0).

`(n >> i) & 1` for checking if ith bit set or not

`n & (n + 1)` clears all trailing ones

`n | (n + 1)` sets last cleared bit

`n & -n` extracts last set bit

9 Bitwise arithmetic identities

`a+b=(a⊕b)+2(a&b)`

`a+b=(a|b)+(a&b)`

`a⊕b=(a|b)-(a&b)`

`a⊕b=(a|b)&~(a&b)` (This is the literal definition of XOR)

```

a+b ≥ a⊕b
a|b = a⊕b + a&b
a⊕(a&b) = (a|b)⊕b
b⊕(a&b) = (a|b)⊕a
(a&b)⊕(a|b) = a⊕b

```

10. Customer comparator

```

struct Comp {
    bool operator()(const pair<int,int>& a, const
pair<int,int>& b) const {
        return a.first + a.second < b.first + b.second;
    }
};

set<pair<int,int>, Comp>;

struct Node { int u, v, cost; // Overloading the <
operator bool operator<(const Node& other) const
{ if (cost != other.cost) return cost < other.cost;
return u < other.u; // Tie-breaker } };

```

GRAPHS

TopoSort

```

int n; // number of vertices
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
        }
    }
    ans.push_back(v);
}

```

```

void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}

```

Bipartite

```

int n;
vector<vector<int>> adj;

vector<int> side(n, -1);
bool is_bipartite = true;
queue<int> q;
for (int st = 0; st < n; ++st) {
    if (side[st] == -1) {
        q.push(st);
        side[st] = 0;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int u : adj[v]) {
                if (side[u] == -1) {
                    side[u] = side[v] ^ 1;
                    q.push(u);
                } else {
                    is_bipartite &= side[u] != side[v];
                }
            }
        }
    }
}

cout << (is_bipartite ? "YES" : "NO") << endl;

```

Dijkstra algorithm

```

const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> &
p) {
    int n = adj.size();

```

```

d.assign(n, INF);
p.assign(n, -1);
vector<bool> u(n, false);

d[s] = 0;
for (int i = 0; i < n; i++) {
    int v = -1;
    for (int j = 0; j < n; j++) {
        if (!u[j] && (v == -1 || d[j] < d[v]))
            v = j;
    }

    if (d[v] == INF)
        break;

    u[v] = true;
    for (auto edge : adj[v]) {
        int to = edge.first;
        int len = edge.second;

        if (d[v] + len < d[to]) {
            d[to] = d[v] + len;
            p[to] = v;
        }
    }
}

```

Bellman ford

```

struct Edge {
    int a, b, cost;
};

int n, m, v;
vector<Edge> edges;
const int INF = 1000000000;

void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    for (int i = 0; i < n - 1; ++i)
        for (Edge e : edges)
            if (d[e.a] < INF)
                d[e.b] = min(d[e.b], d[e.a] + e.cost);
    // display d, for example, on the screen
}

```

Floyd warshall

```

for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}

if neg weights then
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}

```

Cycle detection directed graph

```

int n;
vector<vector<int>>> adj;
vector<char> color;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v) {
    color[v] = 1;
    for (int u : adj[v]) {
        if (color[u] == 0) {
            parent[u] = v;
            if (dfs(u))
                return true;
        } else if (color[u] == 1) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
    }
    color[v] = 2;
    return false;
}

void find_cycle() {
    color.assign(n, 0);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (color[v] == 0 && dfs(v))

```

```

        break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v =
parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);
        reverse(cycle.begin(), cycle.end());

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}

```

Cycle detection undirected graph

```

int n;
vector<vector<int>>> adj;
vector<bool> visited;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v, int par) { // passing vertex and its
parent vertex
    visited[v] = true;
    for (int u : adj[v]) {
        if(u == par) continue; // skipping edge to
parent vertex
        if (visited[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u, parent[u]))
            return true;
    }
    return false;
}

void find_cycle() {
    visited.assign(n, false);
    parent.assign(n, -1);

```

```

    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && dfs(v, parent[v]))
            break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v =
parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}

```

MST (Prims)

```

#include <iostream>
#include <vector>
#include <queue>
#include <limits>

using namespace std;

// Pair representing (weight, destination_node)
typedef pair<int, int> pii;

void primsMST(vector<vector<pii>>& adj, int
startNode, int V) {
    // Min-heap to pick the minimum weight edge
    priority_queue<pii, vector<pii>, greater<pii>>
pq;

    vector<int> key(V, INT_MAX); // Minimum
weight to connect node
    vector<int> parent(V, -1); // To store the
structure of MST
    vector<bool> inMST(V, false); // To track nodes
included in MST

    // Start with the initial node
    pq.push({0, startNode});

```

```

key[startNode] = 0;

int totalCost = 0;

while (!pq.empty()) {
    int u = pq.top().second;
    int weight = pq.top().first;
    pq.pop();

    if (inMST[u]) continue; // Skip if node is
    already in MST

    inMST[u] = true;
    totalCost += weight;

    // Explore neighbors
    for (auto& neighbor : adj[u]) {
        int v = neighbor.first;
        int w = neighbor.second;

        // If v is not in MST and weight of (u,v) is
        smaller than current key
        if (!inMST[v] && w < key[v]) {
            key[v] = w;
            pq.push({key[v], v});
            parent[v] = u;
        }
    }
}

// Print the result
cout << "Edge \tWeight" << endl;
for (int i = 1; i < V; ++i) {
    if (parent[i] != -1)
        cout << parent[i] << " - " << i << " \t" <<
key[i] << endl;
}

cout << "Total Minimum Cost: " << totalCost <<
endl;
}

int main() {
    int V = 6; // Number of vertices
    vector<vector<pii>> adj(V);

    // Adding edges: adj[u].push_back({v, weight})
    adj[0].push_back({1, 2}); adj[1].push_back({0,
2});

```

```

adj[0].push_back({2, 3}); adj[2].push_back({0,
3});
adj[1].push_back({2, 1}); adj[2].push_back({1,
1});
adj[1].push_back({3, 1}); adj[3].push_back({1,
1});
adj[1].push_back({4, 4}); adj[4].push_back({1,
4});
adj[3].push_back({4, 1}); adj[4].push_back({3,
1});
adj[4].push_back({5, 1}); adj[5].push_back({4,
1});

primsMST(adj, 0, V);

return 0;
}

```

Investigation (Dijkstra with states)

```

#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>

using namespace std;

#define ll long long

static const ll MOD = 1e9 + 7;
const ll INF = 1e17;

ll dist[100005];
ll ways[100005];
ll min_f[100005];
ll max_f[100005];
vector<vector<pair<ll, ll>>> adj;

void dijkstra(ll n) {
    // 1. Initialize all arrays
    for (int i = 1; i <= n; i++) {
        dist[i] = INF;
        ways[i] = 0;
        min_f[i] = INF;
        max_f[i] = 0;
    }

    // 2. Base Case: Start at City 1
    dist[1] = 0;
    ways[1] = 1;

```

```

min_f[1] = 0;
max_f[1] = 0;

// pair<price, node>
priority_queue<pair<ll, ll>, vector<pair<ll, ll>>,
greater<pair<ll, ll>>> q;
q.push({0, 1});

while (!q.empty()) {
    ll d_u = q.top().first;
    ll u = q.top().second;
    q.pop();

    // 3. Standard Dijkstra check
    if (d_u > dist[u]) continue;

    for (auto& edge : adj[u]) {
        ll v = edge.first;
        ll weight = edge.second;

        // Case A: Found a strictly better price
        if (dist[u] + weight < dist[v]) {
            dist[v] = dist[u] + weight;
            ways[v] = ways[u];
            min_f[v] = min_f[u] + 1;
            max_f[v] = max_f[u] + 1;
            q.push({dist[v], v});
        }
        // Case B: Found an alternative path with
        // the same minimum price
        else if (dist[u] + weight == dist[v]) {
            ways[v] = (ways[v] + ways[u]) % MOD;
            min_f[v] = min(min_f[v], min_f[u] + 1);
            max_f[v] = max(max_f[v], max_f[u] +
1);
        }
    }
}

cout << dist[n] << " " << ways[n] << " " <<
min_f[n] << " " << max_f[n] << endl;
}

void solve() {
    ll n, m;
    if (!(cin >> n >> m)) return;

    adj.assign(n + 1, vector<pair<ll, ll>>());
    for (ll i = 0; i < m; i++) {

```

```

        ll a, b, c;
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
    }

    dijkstra(n);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    // precompute(); // Not needed for this specific
    // logic, but can keep if used elsewhere
    solve();
    return 0;
}

```

DSU (With CSES Question)

```

#include <bits/stdc++.h>
using namespace std;

class DisjointSet {
public:
    vector<int> parent, sz;
    int numComponents;
    int maxComponentSize;

    DisjointSet(int n) {
        parent.resize(n + 1);
        sz.resize(n + 1, 1); // Initially, each city is a
        // component of size 1
        numComponents = n; // Initially, there are n
        // components
        maxComponentSize = 1; // Largest component
        // starts at 1
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
        }
    }

    int findUPar(int node) {
        if (node == parent[node])
            return node;
        return parent[node] = findUPar(parent[node]);
    }

    void unionBySize(int u, int v) {
        int ulp_u = findUPar(u);

```

```

int ulp_v = findUPar(v);

// If they are already in the same component,
do nothing
if (ulp_u == ulp_v) return;

// Union by size: attach smaller to larger
if (sz[ulp_u] < sz[ulp_v]) {
    parent[ulp_u] = ulp_v;
    sz[ulp_v] += sz[ulp_u];
    maxComponentSize =
max(maxComponentSize, sz[ulp_v]);
} else {
    parent[ulp_v] = ulp_u;
    sz[ulp_u] += sz[ulp_v];
    maxComponentSize =
max(maxComponentSize, sz[ulp_u]);
}

// One successful merge means one less
component
numComponents--;
}
};

int main() {
    // Speed up I/O for CP
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    DisjointSet ds(n);

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        ds.unionBySize(u, v);

        // Print the two required values after each road
        cout << ds.numComponents << " " <<
ds.maxComponentSize << "\n";
    }

    return 0;
}

```

SCC

```

#include <bits/stdc++.h>
using namespace std;

vector<bool> visited;

// DFS to find the finishing order
void dfs(int v, vector<vector<int>> const& adj,
vector<int> &output) {
    visited[v] = true;
    for (auto u : adj[v])
        if (!visited[u])
            dfs(u, adj, output);
    output.push_back(v);
}

// Your provided SCC logic
void strongly_connected_components(int n,
vector<vector<int>> const& adj,
vector<int>
&kingdom_assignment,
int &kingdom_count) {
    vector<int> order;
    visited.assign(n + 1, false);

    // Step 1: First DFS to get finishing times
    for (int i = 1; i <= n; i++)
        if (!visited[i])
            dfs(i, adj, order);

    // Step 2: Reverse the graph
    vector<vector<int>> adj_rev(n + 1);
    for (int v = 1; v <= n; v++) {
        for (int u : adj[v]) {
            adj_rev[u].push_back(v);
        }
    }

    // Step 3: Second DFS on reversed graph in order
of finishing times
    visited.assign(n + 1, false);
    reverse(order.begin(), order.end());

    kingdom_count = 0;
    for (auto v : order) {
        if (!visited[v]) {
            kingdom_count++;
            vector<int> component;
            dfs(v, adj_rev, component);

```



```

        // Assign the current kingdom ID to all
planets in this component
        for (auto u : component) {
            kingdom_assignment[u] =
kingdom_count;
        }
    }
}

int main() {
    // Fast I/O
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<vector<int>>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
    }

    vector<int> kingdom_assignment(n + 1);
    int total_kingdoms = 0;

    strongly_connected_components(n, adj,
kingdom_assignment, total_kingdoms);

    // Output the results
    cout << total_kingdoms << "\n";
    for (int i = 1; i <= n; i++) {
        cout << kingdom_assignment[i] << (i == n ?
"" : " ");
    }
    cout << endl;

    return 0;
}

```